FlavourKit

Recipe & Cook-Together App

Interview & Viva Question Bank

Tech Stack: React 18 · Vite · React Router v6 · PeerJS (WebRTC) · Plain CSS · Vercel

Difficulty: ● Easy ● Medium ● Hard

1. Project Overview & Architecture

#	Question	Level
1	What is FlavourKit and what problem does it solve?	Easy
2	Explain the overall architecture of this project. What is the tech stack used?	Easy
3	Why did you choose React with Vite instead of Create React App?	Medium
4	Why is there no backend or database in this project? What are the trade-offs?	Medium
5	How is data managed in this application? Where is it stored and how is it accessed?	Easy
6	Explain the folder structure of the project and why it is organised that way.	Easy

Model Answers

Q1. What is FlavourKit and what problem does it solve?

- FlavourKit is a recipe browsing website with a live video calling feature.
- It solves the problem of cooking alone by letting two people cook the same recipe together over video.
- Users browse categories, view full recipes, then start a WebRTC video room — the recipe stays visible in a side panel during the call.

Q2. Explain the overall architecture of this project. What is the tech stack used?

- **Frontend:** React 18 (JSX, Hooks, React Router v6)

- **Build tool:** Vite (fast dev server + HMR)
- **Styling:** Plain CSS with CSS variables
- **Data:** Local JSON file (no database)
- **Video:** PeerJS library (WebRTC wrapper)
- **Deployment:** Vercel (static hosting)

Q3. Why did you choose React with Vite instead of Create React App?

- Vite uses native ES modules in development — startup is near instant regardless of project size.
- HMR (Hot Module Replacement) is much faster in Vite than CRA.
- CRA is based on Webpack which bundles everything before serving — slow for large apps.
- `create vite@latest` also has better TypeScript and JSX defaults out of the box.

Q4. Why is there no backend or database in this project? What are the trade-offs?

- **Trade-offs of no backend:** + Zero hosting cost, + No API maintenance, + Instant deployment
- – Data cannot be updated without a redeployment, – No user authentication possible, – All data visible in source code.
- For a recipe showcase with a fixed dataset, a backend adds unnecessary complexity.

Q5. How is data managed in this application? Where is it stored and how is it accessed?

- All recipe and category data lives in `src/data/recipes.json`.
- It is imported directly into components as a JavaScript module: `import data from "../data/recipes.json"`
- No fetch calls or loading states needed — data is available synchronously at render time.

Q6. Explain the folder structure of the project and why it is organised that way.

- `components/` — reusable UI pieces (Slider, CategoryList, RecipeCard)
- `pages/` — route-level components (Home, CategoryPage, RecipePage, Room)
- `data/` — static JSON data source
- `styles/` — single global CSS file with CSS variables
- This separation keeps concerns clear and makes the project easy to navigate.

2. React & Component Design

#	Question	Level
1	What is the difference between a controlled and uncontrolled component in React? Give an example from this project.	Medium
2	Explain how React Router v6 is used in this project. What is the difference between <Route> and <Link>?	Easy
3	What are React Hooks? Name the hooks used in this project and explain each one.	Medium
4	What is useRef and why is it used for video elements instead of useState?	Hard
5	Explain the concept of lifting state up with an example from this codebase.	Medium
6	What is the purpose of the key prop in lists? What happens if you omit it?	Easy
7	Why are the <video> elements always kept in the DOM instead of conditionally rendered?	Hard

Model Answers

Q1. What is the difference between a controlled and uncontrolled component in React? Give an example from this project.

- **Controlled:** Form element whose value is driven by React state. The component owns the value.
- Example: The step-checker in RecipePanel — `checked` state controls which steps appear done.
- **Uncontrolled:** Element manages its own internal state (like a native <video> element).
- The `<video>` srcObject is set imperatively via a ref, making it effectively uncontrolled from React's perspective.

Q2. Explain how React Router v6 is used in this project. What is the difference between <Route> and <Link>?

- React Router renders components based on the current URL without page reloads.
- `<Routes>` + `<Route path="/recipe/:id" element={<RecipePage/>}>` maps URLs to components.
- `<Link to="/category/breakfast">` renders an <a> tag but intercepts clicks to update the URL via the History API (no full page reload).
- `<Route>` defines a mapping; `<Link>` is a navigation trigger.

Q3. What are React Hooks? Name the hooks used in this project and explain each one.

- `useState` — manages local component state (slider index, tab selection, connected status)
- `useEffect` — runs side effects after render (PeerJS setup, media stream init, stream wiring)
- `useRef` — holds mutable values that don't cause re-renders (video elements, stream refs, peer ref)

- `useCallback` — memoises functions to prevent unnecessary re-creation (wireDataConn, sendMediaState)
- `useParams` — reads URL params like `:id` and `:roomId` from React Router

```
useSearchParams` — reads query string params like `?recipe=butter-chicken
```

Q4. What is useRef and why is it used for video elements instead of useState?

- `useRef` holds a reference to a DOM element or a mutable value across renders WITHOUT triggering a re-render when changed.
- If `localVideoRef` were stored in `useState`, every time a stream was attached (`ref.current.srcObject = stream`) React would re-render the component — potentially resetting the video element.
- With `useRef`, we can imperatively set `.srcObject` on the actual DOM node without React interfering.
- The video element needs direct DOM access — refs are the correct React pattern for this.

Q5. Explain the concept of lifting state up with an example from this codebase.

- In RecipePanel, `checked` (step completion) state lives inside the panel component itself — it doesn't need to be shared upward.
- But `remoteMicOff` / `remoteCamOff` live in the parent `Room` component because both the video tile (to show overlay) and the status bar need to read it.
- This is lifting state up — moving state to the lowest common ancestor of all components that need it.

Q6. What is the purpose of the key prop in lists? What happens if you omit it?

- React uses `key` to identify which list items changed, were added, or removed between renders.
- Without it, React falls back to index-based comparison — if you remove the first item, React thinks every item changed, causing unnecessary re-renders or wrong component reuse.
- In this project: `{recipe.ingredients.map((item, i) => <li key={i}>)}` — using index as key is acceptable here since the list never reorders.

Q7. Why are the <video> elements always kept in the DOM instead of conditionally rendered?

- If a `<video>` element is conditionally rendered (`{connected && <video ref={remoteVideoRef}/>}`), it only exists in the DOM when `connected` is true.
- But when the stream arrives, `connected` is still false — so `remoteVideoRef.current` is null and `srcObject` assignment is lost.
- By keeping the video always in DOM and toggling `display:none` via CSS, the ref is always valid and we can set srcObject the moment the stream arrives.

3. WebRTC & PeerJS Video Calling

#	Question	Level
1	What is WebRTC? How does it differ from a regular HTTP connection?	Medium
2	What is PeerJS and why was it chosen over the raw WebRTC API?	Easy
3	Explain the host/guest model used in the Room component. How does a peer decide which role to take?	Hard
4	What is a STUN server and why is it needed for WebRTC?	Medium
5	Why does PeerJS sometimes echo the local stream back as the remote stream? How is this handled?	Hard
6	What is a DataConnection in PeerJS and why is it used here instead of relying on track events?	Hard
7	What happens when getUserMedia() is called? What permissions does it require?	Easy
8	Explain the fallback chain in getBestStream(). Why is this important?	Medium

Model Answers

Q1. What is WebRTC? How does it differ from a regular HTTP connection?

- HTTP is a client-server protocol — data always flows through a server.
- WebRTC is a peer-to-peer protocol — once connected, audio/video flows directly between two browsers with no server in between.
- WebRTC uses UDP (User Datagram Protocol) for low-latency media streaming, rather than TCP.
- It requires a signalling step (via PeerJS cloud server) to exchange connection info — but after that the media is direct.

Q2. What is PeerJS and why was it chosen over the raw WebRTC API?

- PeerJS is a JavaScript library that abstracts the complex WebRTC API into simple `.call()` and `.on("stream")` methods.
- Raw WebRTC requires manually handling: RTCPeerConnection, ICE candidates, SDP offer/answer exchange, media track management.
- PeerJS handles all of this internally and provides a cloud signalling server for free — ideal for a project with no backend.

Q3. Explain the host/guest model used in the Room component. How does a peer decide which role to take?

- When a user enters a room, the app tries to register as the HOST using the fixed peer ID `recipetogether-host-{roomId}`.
- If that ID is already taken (another user is already in the room), PeerJS fires an `unavailable-id` error.

- On this error, the user becomes a GUEST — they create a peer with a random ID and call the host peer ID directly.
- This works because the room URL is shared — both users visit the same `/room/:id` and the ID determines the host peer ID.

Q4. What is a STUN server and why is it needed for WebRTC?

- NAT (Network Address Translation) is used by home routers — your device has a private IP (e.g. 192.168.x.x) that is not reachable from the internet.
- A STUN server (Session Traversal Utilities for NAT) tells each peer what their public IP address is.
- WebRTC uses this info to find a connection path between the two peers.
- Google's free STUN servers (`stun.l.google.com:19302`) are used in this project.

Q5. Why does PeerJS sometimes echo the local stream back as the remote stream? How is this handled?

- PeerJS's MediaConnection fires the `stream` event once when the call connects. In some versions/browsers, it fires twice — first with a copy of the local stream being "reflected" back.
- The guard in `onRemoteStream`: (1) rejects if `remoteStream.id === localStream.id`, (2) rejects if all track IDs in the remote stream match local track IDs.
- This ensures only genuine remote streams (with different track IDs) are accepted and rendered.

Q6. What is a DataConnection in PeerJS and why is it used here instead of relying on track events?

- When a user toggles mic/camera, `track.enabled = false` is a LOCAL operation — WebRTC sends no notification to the other peer.
- `track.onmute` only fires when the remote side actually stops SENDING data (network issue), not when they intentionally mute.
- A DataConnection is a PeerJS data channel — it lets us send arbitrary JSON messages between peers.
- When mic/cam is toggled, we send `{type: "mediaState", micOn: false, camOn: true}` over the data channel — the receiver updates the UI immediately.

Q7. What happens when getUserMedia() is called? What permissions does it require?

- `navigator.mediaDevices.getUserMedia({ video: true, audio: true })` asks the browser for access to the camera and microphone.
- The browser shows a native permission prompt — the user must click "Allow".
- It returns a Promise that resolves to a `MediaStream` object containing video and audio tracks.
- If denied, it throws a `NotAllowedError`. If no device is found, it throws `NotFoundError`.

Q8. Explain the fallback chain in getBestStream(). Why is this important?

- The fallback chain tries: (1) video+audio → (2) audio-only → (3) silent AudioContext stream.
- This ensures the app never crashes even on devices without cameras or when permissions are partially granted.
- Each attempt catches its error and classifies it: "permission", "not-found", "in-use".
- For "permission" errors the chain stops immediately (no point trying audio-only if the user denied all access) and shows a retry banner.

4. CSS & Styling

#	Question	Level
1	What are CSS custom properties (variables)? How are they used in this project?	Easy
2	How is the hero image slider implemented without any library?	Medium
3	What is the difference between position: absolute and position: relative? Give examples from the Room page.	Easy
4	How does the two-column room layout work responsively on mobile?	Medium
5	What is backdrop-filter and where is it used in this project?	Medium

Model Answers

Q1. What are CSS custom properties (variables)? How are they used in this project?

- CSS custom properties are variables defined with `--name: value` and accessed with `var(--name)`.
- In `:root { --pink: #F93481; --font-body: "DM Sans" }`, these values are available everywhere.
- This makes global theme changes easy — changing `--pink` updates every button, badge, and accent in one place.
- It also allows conditional theming (dark mode) by redefining variables inside a class or media query.

Q2. How is the hero image slider implemented without any library?

- The slider uses a flex container (`.slider__track`) whose width is 100% and whose children (`.slide`) each have `min-width: 100%`.
- CSS `transform: translateX(-${current * 100}%)` shifts the track to show the current slide.
- A `transition: transform 0.7s cubic-bezier(...)` creates the smooth animation.
- A `setInterval` in `useEffect` auto-advances every 5 seconds. The cleanup function clears the interval on unmount.

Q3. What is the difference between position: absolute and position: relative? Give examples from the Room page.

- `position: relative` establishes a positioning context for absolutely-positioned children. The element stays in normal flow.
- `position: absolute` removes the element from normal flow and positions it relative to the nearest `relative` ancestor.
- In `.video-container { position: relative }` — the container stays in the grid.
- `.video-container__label { position: absolute; bottom: 10px; left: 10px }` — the label sits over the video without affecting layout.
- `.video-status-bar { position: absolute; top: 10px; right: 10px }` — the mic/cam icons float over the video corner.

Q4. How does the two-column room layout work responsively on mobile?

- On desktop: `.room-layout { display: grid; grid-template-columns: 1fr 380px }` — video left, recipe right.
- At 1024px breakpoint: `grid-template-columns: 1fr` switches to single column, recipe panel stacks below.
- The recipe panel gets `height: auto; max-height: 480px` so it doesn't take over the screen on mobile.
- At 768px: videos also stack vertically (`grid-template-columns: 1fr`).

Q5. What is backdrop-filter and where is it used in this project?

- `backdrop-filter: blur(8px)` applies a Gaussian blur to everything behind the element.
- Used on `.slider__arrow` buttons — the frosted-glass arrows over the hero image.
- Also used on `.video-container__label` and `.vstatus-icon` — labels float over video with a blurred background.
- Requires `background: rgba(...)` with some transparency, otherwise there is nothing to blur through.

5. Routing & Deployment

#	Question	Level
1	What is client-side routing? How is it different from server-side routing?	Easy
2	Why did friend's browser get a 404 when opening the room link on Vercel? How was it fixed?	Medium
3	How is the recipe passed to the friend's Room page? Why can't sessionStorage be used?	Medium
4	What does the vercel.json file do? Explain each field.	Easy
5	What is the difference between npm run dev and npm run build?	Easy

Model Answers

Q1. What is client-side routing? How is it different from server-side routing?

- **Server-side routing:** The browser requests `/recipe/carbonara` from the server, which returns an HTML page for that route.
- **Client-side routing:** The server always returns the same `index.html`. JavaScript (React Router) reads the URL and renders the correct component — no server round-trip needed.
- This is faster (no full page reload) but requires the server to be configured to always serve `index.html` for all routes.

Q2. Why did friend's browser get a 404 when opening the room link on Vercel? How was it fixed?

- Vercel serves static files from `dist/`. When the friend visits `/room/abc123`, Vercel looks for a real file at that path — there isn't one, so it returns 404.
- Fix: `vercel.json` with `"rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]`
- This tells Vercel to serve `index.html` for ALL paths, letting React Router handle the URL client-side.

Q3. How is the recipe passed to the friend's Room page? Why can't sessionStorage be used?

- `sessionStorage` is per-browser and per-tab — it never leaves the user's device. The friend's browser has no access to the host's sessionStorage.
- Fix: Encode the recipe ID in the URL itself: `/room/abc123?recipe=butter-chicken`
- When the host copies and shares the invite link, the `?recipe=` parameter travels with it.
- The friend's browser reads it with `useSearchParams().get("recipe")` — same URL, same recipe, zero server needed.

Q4. What does the vercel.json file do? Explain each field.

- `"buildCommand": "npm run build"` — tells Vercel how to build the project.
- `"outputDirectory": "dist"` — tells Vercel where Vite puts the built files.

- `"framework": "vite"` — hints to Vercel which framework is used for optimisations.
- `"rewrites": [...]` — redirects all routes to index.html for SPA (Single Page Application) support.

Q5. What is the difference between npm run dev and npm run build?

- `npm run dev` starts the Vite development server with HMR (Hot Module Replacement) — fast reloads on file save, source maps enabled, no minification.
- `npm run build` compiles everything into static files in `dist/`: JS is bundled + minified, CSS is optimised, images are copied from `public/`.
- The `dist/` folder is what gets deployed to Vercel — not the source code.

6. Advanced & Tricky Questions

#	Question	Level
1	What is a race condition? Give an example from the video call code and explain how it was solved.	Hard
2	Why does mobile Safari block audio autoplay? How would you handle it?	Hard
3	What is the difference between track.enabled = false and track.stop()?	Hard
4	Explain the concept of a memory leak in React. Is there any risk of one in the Room component?	Hard
5	If you had to add a backend to this project, what would you add and why?	Hard
6	How would you scale this app to support rooms with more than 2 users?	Hard

Model Answers

Q1. What is a race condition? Give an example from the video call code and explain how it was solved.

- A race condition occurs when the outcome depends on the timing of two asynchronous events.
- Example: PeerJS fires `call.on("stream")` with the remote stream BEFORE React has rendered the `<video>` element — `remoteVideoRef.current` is null, so `srcObject` assignment is silently lost.
- Solution: Keep `<video>` always in the DOM (using `display:none` CSS) so the ref is always valid. Set `srcObject` directly in the stream callback — no waiting for render needed.

Q2. Why does mobile Safari block audio autoplay? How would you handle it?

- Mobile browsers enforce an autoplay policy: media with audio cannot start without a direct user gesture (tap/click).
- Even with `autoPlay` on the element, `srcObject` assignment via JavaScript is treated as automatic and may be blocked.
- Handling: After setting srcObject, call `.play()` explicitly. If it rejects (Promise catch), show a "Tap to hear audio" button. When the user taps it, call `.play()` again — this time triggered by a user gesture, which the browser allows.

Q3. What is the difference between track.enabled = false and track.stop()?

- `track.enabled = false` — pauses the track locally. The track object still exists, the stream is still connected. Setting `.enabled = true` re-enables it instantly. Audio/video just goes silent/black on the receiver side.
- `track.stop()` — permanently ends the track. The hardware (camera/mic) is released. Cannot be re-enabled. The user's camera light turns off.
- In the toggle buttons, `enabled` is used so the user can turn camera/mic back on. `stop()` is only called in `endCall()` when the call is permanently ended.

Q4. Explain the concept of a memory leak in React. Is there any risk of one in the Room component?

- A memory leak in React occurs when a component is unmounted but still has active callbacks that try to call `setState` or hold references to DOM nodes.
- Risk in Room: The `useEffect` starts async operations (getUserMedia, PeerJS) that have callbacks (`.on("stream")`, `.on("call")`).
- If the component unmounts before these resolve (user navigates away), callbacks could fire and try to call `setConnected(true)` on an unmounted component.
- Fix: The `useEffect` return function (cleanup) calls `localStream.getTracks().forEach(t => t.stop())` and `peer.destroy()` — which removes all PeerJS event listeners and stops the stream.

Q5. If you had to add a backend to this project, what would you add and why?

- **User accounts** — recipes saved to favourites, call history.
- **Signalling server** — replace PeerJS cloud with a custom WebSocket server for more control and to support more than 2 users.
- **Image storage** — upload user-submitted recipe photos to an S3 bucket or Cloudinary.
- **Database** — store recipes in MongoDB/PostgreSQL so they can be added/edited without redeployment.
- **API** — REST or GraphQL endpoint to serve recipes, supporting search, filtering, and pagination.

Q6. How would you scale this app to support rooms with more than 2 users?

- Current PeerJS setup is 1-to-1 (one host calls one guest).
- For 3+ users: switch to a **mesh** topology — each peer connects to every other peer (N*(N-1)/2 connections). Works for small groups (3–4 people) but doesn't scale.
- For larger groups: use a **SFU** (Selective Forwarding Unit) media server like **mediasoup** or **LiveKit**. Each peer sends one stream to the server, which forwards it to everyone else.
- The signalling layer would also need to change — a WebSocket server to coordinate who is in the room.

Quick Reference — Cheat Sheet

Concept	One-Line Answer	Where Used
useState	Stores reactive data that triggers re-render on change	Slider index, connected status, mic/cam toggles
useEffect	Runs side-effects after render (setup, teardown)	PeerJS init, media stream setup, cleanup on unmount
useRef	Holds a mutable value without re-rendering	localVideoRef, remoteVideoRef, peerRef, localStreamRef
useCallback	Memoises a function reference across renders	wireDataConn, sendMediaState
useParams	Reads URL segment params like :id	roomId in Room.jsx, recipe id in RecipePage.jsx
useSearchParams	Reads URL query params like ?recipe=	Reading recipe id in Room.jsx from invite link
WebRTC	Peer-to-peer audio/video in the browser, no server needed for media	Room.jsx — entire video call
PeerJS	WebRTC wrapper with simple API and free signalling server	Room.jsx — peer.call(), peer.on("call")
DataConnection	PeerJS data channel for sending arbitrary JSON messages	Broadcasting mic/cam state between peers
STUN server	Tells each peer its public IP so they can find each other	peerConfig.config.iceServers in Room.jsx
vercel.json rewrites	Serves index.html for all routes — fixes 404 on refresh/direct link	Vercel deployment configuration
track.enabled	Pauses a media track without stopping it (reversible)	toggleMic(), toggleCam() in Room.jsx
track.stop()	Permanently stops a track and releases the hardware	endCall() in Room.jsx
sessionStorage	Browser-local key/value store — NOT shared between users	Was incorrectly used; replaced with URL params

Good luck with your interview! □